

PredictiX

Predictive Maintenance Platform for Fleet and Warehouse Operations

Technical & Theoretical Documentation

Document type	System documentation and engineering assessment
Version	1.0
Date	13 August 2026
Author	Dinusha Ekanayake
Backend	FastAPI · SQLAlchemy · Supabase PostgreSQL
Frontend	Next.js 16 · React 19 · TypeScript
Status	Operational, with defects recorded in §12

Every figure in this document was measured against the running system and its database on the stated date. Where a value could not be verified it is marked as unverified rather than estimated.

Table of Contents

1. Executive Summary	3
2. System Architecture	3
3. Data Model and Live Data Profile	4
4. Machine Learning Subsystem — Theory	6
5. Machine Learning Subsystem — Implementation	7
6. The Decision Layer	13
7. Language Model and Retrieval Subsystem	14
8. API Surface	15
9. Frontend Application	16
10. Security Model	16
11. Testing and Continuous Integration	17
12. Known Defects and Open Problems	19
13. Technical Debt	21
14. Recommendations	22
Appendix A — Verification Method	22
Appendix B — Complete Technology Stack	24
Appendix C — Complete Endpoint Reference (140 endpoints)	26
Appendix D — Data Flows	34
Appendix E — Applied Optimisations	36
Appendix F — Frontend Structure in Detail	38
Appendix G — Backend Structure in Detail	41

1. Executive Summary

PredictiX is a predictive-maintenance platform for a Sri Lankan logistics fleet. It ingests vehicle telemetry, predicts whether each asset will require maintenance within a 30-day horizon, estimates when, estimates what the repair will cost, and forecasts remaining life for five individual components. Results are presented through a role-scoped web application and are queryable in natural language through a retrieval-augmented assistant.

The platform is operational. It manages 850 assets across three warehouses, serves 1,337 user accounts, and holds five years of sensor history. Four trained models are deployed and a fifth subsystem provides survival analysis.

1.1 Verified system scale

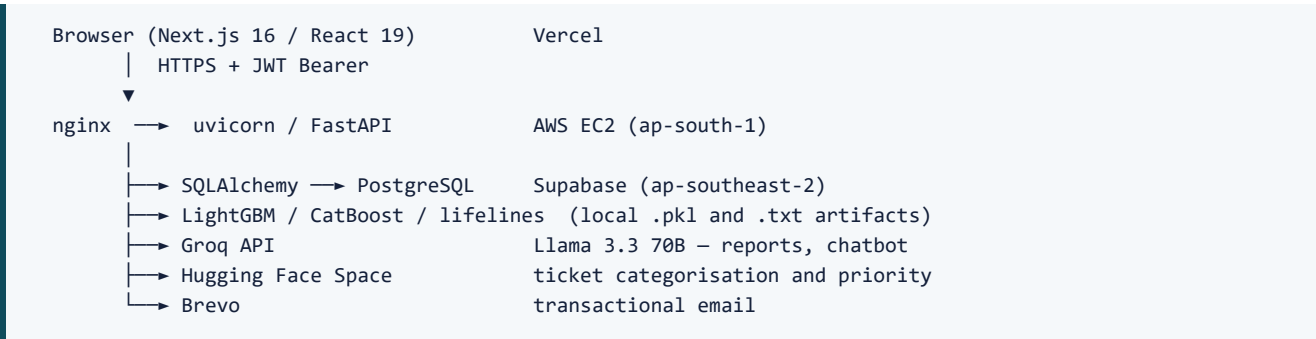
Entity	Rows	Notes
Warehouses	3	Colombo 400, Galle 250, Badulla 200 assets
Assets	850	Vehicles and materials-handling equipment
User profiles	1,337	Across admin, super admin and user roles
Sensor readings	51,850	10 Aug 2021 – 10 Aug 2026
Maintenance events	15,765	Planned and unplanned
Tickets	935	With AI categorisation and prioritisation
Batch predictions	850	Exactly one current row per asset
Departments	15	

1.2 Assessment

The system's engineering strengths are a single authoritative prediction row per asset, consistent role-based scoping across the API, and a decision layer that reconciles disagreeing model outputs rather than averaging them away. Its weaknesses are concentrated in three areas: an unscoped warehouse report, two external dependencies that have silently disappeared, and a deployment that drifts behind the main branch. All are documented in §12 with measured impact.

2. System Architecture

2.1 Component topology



The application process is single-instance. An APScheduler job inside that process re-runs predictions for every asset on a 24-hour interval; it calls the batch service directly as Python rather than over HTTP.

Architectural issue — cross-region latency

The application server is in AWS `ap-south-1` (Mumbai) and the database is in Supabase `ap-southeast-2` (Sydney). Measured round-trip time is **154 ms from EC2** and 231 ms from the development machine. Because request cost is dominated by round-trips rather than query complexity, endpoints returning zero rows still take 900–1,700 ms. Instrumentation confirmed only 2–4 SQL statements per request, so this is latency, not N+1. Co-locating the two would reduce request latency by roughly 3–5×.

2.2 Technology versions

Layer	Technology	Version
API framework	FastAPI	0.128.1
ORM	SQLAlchemy	2.0.46
Gradient boosting	LightGBM	4.6.0
Cost model	CatBoost	1.2.10
Survival analysis	lifelines	0.30.3
Frontend framework	Next.js	16.2.9
UI library	React	19.2.3
Charting	Recharts	3.6.0
Styling	Tailwind CSS	4.1.18

3. Data Model and Live Data Profile

3.1 Core entities

Table	Purpose	Key relationships
<code>warehouses</code>	Physical sites; the unit of access scoping	Parent of assets, profiles, tickets
<code>profiles</code>	Users, roles, department and warehouse assignment	→ warehouse, department
<code>assets</code>	Vehicles and equipment with static and derived attributes	→ warehouse, assigned user
<code>sensor_readings</code>	Component health percentages and operating telemetry	→ asset
<code>maintenance_events</code>	Completed and scheduled service records	→ asset, performing user
<code>tickets</code>	Fault reports with model-assigned category and priority	→ asset, warehouse, users
<code>pdm_batch_predictions</code>	Current model output, one row per asset	→ asset
<code>model_registry</code>	Which model version is live, with its metrics	—

3.2 The single-prediction-row design

`pdm_batch_predictions` holds exactly one current row per asset, upserted by both the scheduled run and the manual refresh. Every consumer — dashboard, asset panel, PDF reports and the chatbot — reads that same row. This is the system's most important consistency property: two screens cannot disagree about an asset because there is only one place the answer comes from.

An earlier design used an append-only `asset_failure_predictions` history table requiring a `DISTINCT ON` to deduplicate. That table is now empty and unwritten; the endpoints reading it were removed during this review.

3.3 Health band definition

Asset condition is expressed as one of five bands, defined once in `app/services/health_bands.py` and enforced at schema level by the `asset_health_band` PostgreSQL enum.

Band	Score range	Assets	Share
excellent	≥ 60	62	7.3%
good	50 – 59	151	17.8%
moderate	38 – 49	236	27.8%
poor	25 – 37	136	16.0%
critical	< 25	265	31.2%

Why the thresholds are not 90/75/60/40

The health score is the mean of five component health percentages *minus* a penalty for failure probability and service urgency. It is therefore systematically lower than raw component health, and peaks at approximately 79 across the real fleet. Bands calibrated on an assumed 0–100 spread produced an unreachable top band and placed 91% of a normally-worn fleet in the two worst bands. The thresholds above are calibrated against the observed distribution so that every band is reachable and each one carries information.

4. Machine Learning Subsystem — Theory

PredictiX answers four distinct questions about an asset, and deliberately uses a different model family for each because the questions have different mathematical shapes.

Question	Formulation	Model family
Will it need maintenance within 30 days?	Binary classification	Gradient-boosted trees (LightGBM)
How many days until it does?	Regression on a duration	Gradient-boosted trees (LightGBM)
What will the repair cost?	Regression with an interval	CatBoost point + LightGBM quantiles
How long will each component last?	Time-to-event with censoring	Weibull AFT (lifelines)

4.1 Why gradient-boosted trees for the primary task

The feature space is 58 mixed numeric and categorical variables — vehicle type, fuel type, transmission, route profile, five component health percentages, temperatures, vibration, load and service history. Relationships are non-linear and interacting: high vibration matters more on an old vehicle with worn brakes than on a new one. Tree ensembles capture such interactions without manual feature crossing, handle mixed types natively, and are robust to unscaled features.

LightGBM specifically was chosen for its native handling of two properties of this data. First, missing values are treated as an informative branch direction rather than requiring imputation — a sensor that has never reported is different from one reporting zero. Second, categorical features are split on directly using the trained category vocabulary, avoiding one-hot expansion across 13 categorical columns.

4.2 Why survival analysis for component life

Component remaining-life cannot be treated as ordinary regression because the data is *censored*: for most assets, the component has not failed yet, so the true time-to-failure is unknown — all that is known is that it exceeds the observation window. Discarding those rows biases the estimate downward; treating the censoring time as the failure time biases it in a different direction.

Survival analysis models the probability that a component survives beyond time t , and correctly incorporates censored observations as partial information. The Accelerated Failure Time formulation with a Weibull baseline was chosen because it produces directly interpretable output: covariates multiply the expected lifetime, so the model yields a median survival time and quantile bounds rather than only a hazard ratio.

Theoretical consequence of low event rates

Event rates in this dataset are low (1.9%–17.1% depending on component). With few observed failures the Weibull fit is dominated by censored observations, which flattens the survival curve and pushes median estimates toward the upper limit of the trained horizon. This is a property of the data, not a coding defect, but it means the median-days figures should be read as ordering information — which component fails first — rather than as calendar promises.

4.3 Why the health score is not a model

The health score is a weighted composite of five component readings minus a penalty term. It is deliberately *not* a trained model, because no ground truth for "overall asset health" exists to train against. The interface labels it

"Composite indicator" for this reason, and the decision layer treats it as a second opinion rather than as evidence of equal weight to the classifier.

5. Machine Learning Subsystem — Implementation

5.1 Deployed models

Model	Version	Artifact	Purpose
PdM classifier	v7	<code>predictix_pdm_classifier_v7.txt</code>	Maintenance required within 30 days
PdM regressor	v7	<code>predictix_pdm_regressor_v7.txt</code>	Days until maintenance
Breakdown cost	v5	<code>predictix_breakdown_cost_model_v5.pkl</code>	Repair cost with 80% interval
Survival bundle	v3.0	<code>survival_bundle.pkl</code>	Five per-component Weibull AFT models

5.2 Measured model performance

All values below are read from each model's own training log, not from recollection.

Classifier (v7)

Metric	Value	Interpretation
Test ROC AUC	0.9608	Strong separation between classes
Test PR AUC	0.9301	Robust under class imbalance
Operating threshold	0.52	Tuned for maximum F1
Cost-optimal threshold (5:1)	0.20	Available but not currently used
Features	58 (13 categorical)	

The threshold is read at startup from `classifier_v7_decision_log.json` rather than hardcoded, so retraining with a different operating point does not require a code change. The 5:1 cost-optimal threshold reflects an assumption that a missed failure costs five times a false alarm; adopting it would raise recall at the expense of precision. That decision has not been taken.

Regressor (v7)

Metric	Value
Test MAE	13.28 days
Test R ²	0.814

A mean absolute error of 13 days over a 365-day prediction range is the reason the decision layer refuses to present a hard calendar date unless the classifier independently agrees action is needed soon (§6).

Cost model (v5)

Metric	Value
Training rows	19,580
Calibrated PICP (80% target)	0.8122
Fleet mean cost	LKR 51,870.86

Prediction interval coverage probability of 0.8122 against an 80% target means the interval is honestly calibrated: approximately 81% of true costs fall inside the stated range. The model predicts `log1p(cost / fuel_price)` and rescales by the asset's own fuel price, which normalises for fuel-price inflation across the five-year training window.

Survival models (v3.0)

Component	Train rows	Events	Event rate	Test concordance
brake	77,521	3,011	3.9%	0.788
tire	77,521	1,904	2.5%	0.786
battery	77,521	1,584	2.0%	0.776
oil	77,521	13,273	17.1%	0.796
hydraulic	77,521	1,487	1.9%	0.892

Concordance between 0.78 and 0.89 indicates the models order assets by risk substantially better than chance (0.5). Regularisation penalizer is 0.02.

5.3 Inference pipeline

- 1. Load** `LgbModelBundle` wraps each booster, recovering feature names, categorical columns and the training category vocabulary from `Booster.pandas_categorical`.
- 2. Features** `_build_feature_dict` flattens the asset row plus its latest sensor reading into the 58-feature schema.
- 3. Frame** `build_frame` coerces numerics to float (missing → NaN, handled natively) and casts categoricals to the exact training vocabulary. Unseen categories become NaN rather than raising.
- 4. Classify** One batched booster call → `failure_probability`, and `maintenance_required = probability ≥ 0.52`.
- 5. Regress** Days until maintenance, clipped to [1, 365], converted to a calendar date. SHAP contributions via native `pred_contrib` give the top drivers.
- 6. Health** Mean of five component percentages minus failure and urgency penalties.
- 7. Cost** CatBoost point estimate plus LightGBM q10/q90 bounds, widened to contain the point estimate.
- 8. Decide** `build_decision` reconciles the signals into one tier.
- 9. Persist** Upsert one row into `pdm_batch_predictions`.

5.4 Failure handling

The pipeline distinguishes a failed prediction from a confident benign one. If the batched classifier call fails it retries row by row; if a row still fails it records `NULL`, not a fabricated probability. If SHAP extraction fails it falls back to a plain prediction and returns empty explanations rather than invented ones. If the cost model is unavailable it returns `NULL` for all three cost fields rather than substituting a formula.

Design principle

Throughout the prediction path, the absence of a value is represented as `NULL` and surfaced in the interface as a missing state. This is a deliberate constraint: a fabricated number stored alongside real ones cannot be distinguished later, and a maintenance decision made on a guessed input is worse than one deferred for lack of data.

5.5 Complete feature schema

Both v7 models consume an identical 58-feature vector: 45 numeric and 13 categorical. The list below is read directly from the deployed boosters' `feature_name()` metadata.

Categorical features (13)

Feature	Meaning
<code>vehicle_type</code>	Asset class, e.g. delivery van, forklift, reach truck
<code>vehicle_role</code>	Operational duty assigned to the asset
<code>make_model</code>	Manufacturer and model as one token
<code>fuel_type</code>	Diesel, petrol, electric, hybrid
<code>transmission</code>	Manual or automatic
<code>route_type</code>	Dominant route profile
<code>cargo_type</code>	Typical load carried
<code>operating_shift</code>	Day, night or mixed operation
<code>maintenance_priority</code>	Assigned service priority band
<code>service_provider_type</code>	In-house workshop or external vendor
<code>last_service_type</code>	Nature of the most recent service
<code>parts_replaced_last_service</code>	Components replaced at last service
<code>major_component_replaced</code>	Most recent major replacement

Numeric features (45), grouped by what they measure

Group	Features
Identity and age	manufacture_year , vehicle_age_years , payload_capacity_kg
Cumulative usage	odometer_km , engine_hours_total , lifetime_service_count , lifetime_breakdown_count
Recent activity (30 d)	distance_last_30d_km , operating_hours_last_30d , idle_hours_last_30d , trip_count_30d , avg_trip_distance_km
Load and duty stress	avg_payload_kg , payload_utilization_pct , overload_events_30d , start_stop_burden_30d
Route composition	rough_road_pct , urban_route_pct , port_route_pct
Environment	ambient_temp_avg_c , ambient_humidity_avg_pct , rainfall_mm_30d
Component health	oil_life_pct , brake_health_pct , tire_health_pct , battery_health_pct , hydraulic_health_pct
Mechanical telemetry	engine_temp_avg_c , coolant_temp_max_c , vibration_rms_mm_s , tire_pressure_psi , battery_voltage_v , fuel_rate_lph , fuel_efficiency_km_per_l
Service recency	days_since_last_service , mileage_since_last_service_km , engine_hours_since_last_service , maintenance_cost_last_service_lkr , is_home_warehouse_service
Fault signals	active_fault_code_count , sensor_fault_flag , downtime_hours_last_90d
Seasonality	month , year — derived from the snapshot date, not stored
Economic	fuel_price_lkr_per_l — also the cost model's normalisation base

Why month and year are computed, not stored

Sri Lanka's monsoon cycle materially affects wear: `rainfall_mm_30d` , `rough_road_pct` and corrosion-driven battery degradation are all seasonal. Passing the snapshot month lets the trees learn that seasonality directly. They are derived from `snapshot_date` at inference time so a prediction made today uses today's season, never the season the row was written.

5.6 Training data

Property	Value
Dataset	srilanka_single_warehouse_vehicle_maintenance_dataset_v11_realistic.csv
Rows	96,827
Columns	69 (58 used as features)
Survival split	77,521 train / 19,306 test
Cost model rows	19,580
Scope	Single Colombo warehouse, extended to three sites in production

The production fleet was generated by resampling real per-asset trajectories from this dataset, re-dating them to end at the present, redrawing seasonal variables from month-conditional pools, and applying regional conditioning clipped to the bounds observed in v11. This preserves the joint distribution the models were trained on while producing three geographically distinct warehouses.

5.7 Batch pipeline internals

The batch service processes the whole fleet in one pass rather than per asset, which matters because model invocation has fixed overhead and the database is remote.

```
run_batch_for_all_assets(db, clf_model, reg_model, ...)
|
| 1. Load active assets                1 query
| 2. Load latest sensor reading per asset 1 query (LATERAL join)
| 3. Build 850 feature dicts            in memory
| 4. clf_model.build_frame(dict)        one DataFrame
| 5. predict_proba_positive(df)         ONE booster call, 850 rows
|   └─ on failure → per-row retry → NULL on persistent failure
| 6. reg_model.predict(df)              ONE booster call
| 7. shap_top_factors(df)               native pred_contrib
| 8. per asset: _compute_health_score
| 9. per asset: _estimate_cost (cost bundle) → NULL if unavailable
|10. per asset: build_decision          tier + display copy
|11. bulk upsert into pdm_batch_predictions
|12. maintain assets.health_band from the same score
```

Steps 5 and 6 are single vectorised calls for the entire fleet. Step 12 is what keeps the assets list and the prediction row in agreement by construction rather than by coincidence.

5.8 Cost model mechanics

The cost model is a two-part construction. A CatBoost regressor produces the point estimate; two LightGBM quantile regressors (q10 and q90) produce the interval, which is then widened by a conformal correction term `conformal_q_hat` calibrated on held-out data to achieve the stated 80% coverage.

```
point = expm1(max(0, catboost.predict(X))) × fuel_price
lo    = max(0, expm1(max(0, q10.predict(X))) × fuel_price - conformal_q_hat)
hi    =          expm1(      q90.predict(X)) × fuel_price + conformal_q_hat

lo = min(lo, point)      # interval must contain its own estimate
hi = max(hi, point)
```

The final two lines were added during this review. The point estimate and the bounds come from different model families and nothing in training constrains them to agree; on live data roughly half the fleet returned an interval that excluded its own point estimate. The correction widens the interval rather than moving the estimate, so the number each model produced is preserved.

SHAP contributions are computed in log-ratio space. The API exposes `relative_impact` as a percentage share of total absolute SHAP, plus a direction, and deliberately does not expose the raw `sv_log` value, which is not denominated in rupees and would be misread as one.

6. The Decision Layer

The classifier, regressor and health score each answer a different question. If surfaced independently they produce contradictory interfaces — "maintenance not required" printed beside a specific service date. The decision layer exists to reconcile them into one recommendation, and it makes the classifier the gatekeeper: the regressor's date is presented as a commitment only when the classifier independently agrees action is near.

6.1 Tier derivation

```

classifier tier      health tier (via shared bands)
p ≥ 0.52   → urgent    excellent, good → healthy
p ≥ 0.25   → watch     moderate, poor  → watch
otherwise  → healthy    critical      → urgent

combined:
classifier urgent                → urgent
classifier healthy AND health urgent → conflict
either says watch                → watch
both healthy                     → healthy

```

The `conflict` tier is the design's most deliberate choice. When the trained classifier says an asset is fine but its measured component health is in the critical band, averaging the two would hide the disagreement. Surfacing it as a distinct tier routes the asset to a human instead.

6.2 Presentation modes

Tier	Display mode	What the user sees
urgent	<code>date</code>	A specific service date
watch	<code>soft_estimate</code>	"Inspect soon, estimated within ~N days"
conflict	<code>date</code> or <code>soft_estimate</code>	Flagged for manual review
healthy	<code>horizon</code> or <code>soft_estimate</code>	"No maintenance expected in ~N months"

This is why regressor MAE of 13 days does not mislead: a hard date is shown only where the classifier has independently confirmed near-term action, and elsewhere the language is explicitly approximate.

6.3 Current tier distribution

Tier	Assets	Share	Meaning
watch	380	44.7%	Monitor, inspect when convenient
urgent	255	30.0%	Schedule maintenance now
healthy	207	24.4%	Continue monitoring
conflict	8	0.9%	Signals disagree, human review

The 30% urgent rate is consistent with the 29% positive rate the v7 classifier was trained on, which is a useful sanity check that the deployed threshold is behaving as it did in training.

7. Language Model and Retrieval Subsystem

7.1 Components

Function	Provider	Model	Status
Warehouse report narrative	Groq	llama-3.3-70b-versatile	Operational
Chatbot reasoning	Groq	llama-3.3-70b-versatile	Operational
Ticket categorisation	Hugging Face Space	XGBoost classifier	Operational
Ticket priority	Hugging Face Space	XGBoost classifier	Operational
Knowledge embeddings	Hugging Face API	all-MiniLM-L6-v2	Operational
Asset summaries	Hugging Face Space	Seq2Seq	Space returns 404
Ticket summaries	Hugging Face Space	Seq2Seq	Space returns 404

7.2 Retrieval-augmented report generation

1. `build_warehouse_context(db, warehouse_id)`
queries live PostgreSQL for fleet composition, health distribution, maintenance history, ticket state, cost aggregates
2. KB vector store (TF-IDF over 15 documents: ISO 55000, SMRP benchmarks, OEM service intervals) retrieves passages relevant to the query
3. KB annotator applies deterministic annotations – SHAP thresholds, health-band interpretations, benchmark alerts – computed in code, not asked of the model
4. Prompt assembly injects the SQL figures and the KB context together, with explicit instructions to reuse the supplied numbers verbatim
5. Groq Llama 3.3 70B produces the narrative sections

The retrieval layer uses TF-IDF rather than dense embeddings. For a fixed corpus of 15 domain documents with distinctive vocabulary this is adequate and avoids an external dependency on the critical path. The schema notes pgvector is available in Supabase should dense retrieval be adopted later.

Numerical safety in generated text

Every quantitative claim in a generated report is computed in Python and passed into the prompt. The prompt instructs the model to reuse those figures verbatim and forbids introducing a different number for the same quantity. This is why KPI cards and narrative text agree; the model is writing prose around pre-computed values, not deriving statistics.

7.3 Chatbot

The assistant answers questions over fleet data by generating SQL against an allow-listed set of tables and columns, executing it, and summarising the result. It also receives `frontend_context` — cached dashboard and asset snapshots from the browser's local storage — so it can answer about what the user is currently looking at.

Resolved during this review

Those cached snapshots were previously cleared only on explicit logout. On a shared machine the next person to sign in inherited the previous user's figures as chatbot context, and a super admin switching warehouses continued answering from the site they had left. Cache clearing now also runs on session establishment.

8. API Surface

8.1 Composition

Measure	Count
HTTP endpoints (method + path)	140
Unique paths	≈110
WebSocket routes	1 — <code>/ws/notifications/{user_id}</code>
GET / POST / PUT / DELETE / PATCH	69 / 43 / 13 / 13 / 2
Routers	29

The surface was reduced from 172 to 140 endpoints during this review by removing seven router families with no callers in either repository, and seven prediction endpoints whose backing tables were empty and whose own OpenAPI summaries read "[Deprecated — always 404s]".

8.2 Principal routers

Router	Endpoints	Responsibility
<code>/assets</code>	13	Fleet CRUD, filtering, stats, analytics, images
<code>/tickets</code>	14	Ticket lifecycle, AI categorisation and priority
<code>/predictions</code>	7	Raw model inference and cost lookup
<code>/batch-predictions</code>	4	Read and trigger the PdM pipeline
<code>/profiles</code>	9	Self-service profile, assets, stats, colleagues
<code>/user</code>	9	User-role ticket views
<code>/warehouse-dashboard</code>	7	Warehouse KPIs, survival summary, reports
<code>/admin-dashboard</code>	1	Aggregated operations dashboard payload
<code>/notifications</code>	7	In-app notification lifecycle
<code>/survival</code>	2	Per-component remaining-life forecasts
<code>/chatbot</code>	2	Natural-language assistant

8.3 Response conventions

Endpoints return typed Pydantic models. Nullable numeric fields are genuinely nullable end to end: `health_score`, `estimated_cost_lkr`, `min_cost_lkr`, `max_cost_lkr` and `healthPercent` are `NULL` when unknown rather than defaulted, and the frontend types mirror that with `number | null`.

9. Frontend Application

9.1 Structure

Next.js 16 App Router with route groups separating the three audiences.

```
src/app/
  (auth)/login          login, Google OAuth callback
  (admin)/admin/        dashboard, assets, tickets, users, warehouse, settings
  (user)/user/          dashboard, assets, tickets, profile, settings
  help-desk, privacy-policy, terms-of-service

src/components/        admin/, user/, navigation/, chat/, ui/ (shadcn primitives)
src/lib/               apiClient, authService, healthBands, service modules, PDF export
src/proxy.ts           Next 16 middleware convention – loaded by the framework
```

9.2 Data access

All requests pass through `apiClient`, which attaches the JWT and raises a typed `ApiError` carrying the HTTP status. Service modules wrap endpoints per domain. The asset list keeps a 15-second client-side cache keyed by query string so paging back and forth reuses a response.

9.3 Shared health-band definition

`src/lib/healthBands.ts` mirrors the backend module and is the single frontend source for band thresholds and colours. Before it existed the UI carried three independent scales; two of them used an `>= 80` cut-off for "healthy" that no asset could reach, so those components rendered amber or red permanently.

10. Security Model

10.1 Authentication

Credentials are verified against a bcrypt hash stored in `profiles.meta.password_hash`. A JWT is issued carrying the subject and, for super admins, the warehouse selected at login. Super admins complete a two-step flow: authenticate, then select a warehouse, which yields the usable token.

Two credential stores exist

Supabase `auth.users` and `profiles.meta.password_hash` are separate. `POST /auth/login` checks only the latter. An account can therefore exist and authenticate through the Supabase Admin API while being unable to sign in to the application. Any account-provisioning path must write the profile hash.

10.2 Authorisation

`get_current_user` decodes the JWT and re-reads the profile on every request. This costs one database round-trip per call but means deactivating a user takes effect immediately rather than when their token expires up to 24 hours later. The trade was reviewed and retained.

Role	Scope
user	Own profile, assigned assets, own and same-warehouse tickets
admin	Everything within their assigned warehouse
super_admin	Any warehouse, selected at login and carried in the token

10.3 Controls verified during this review

- Model-inference and external-proxy endpoints restricted to administrators, preventing any of 1,337 accounts from exhausting third-party quota
- Notification preferences scoped to the caller for create, read and delete
- Ticket status history requires a ticket id and checks visibility, closing a cross-warehouse enumeration path
- Notification enums constrained at the schema boundary so invalid input yields 422 rather than a database error surfaced as 500
- Every GET endpoint probed across all three roles: no server errors, no cross-warehouse leakage

Operational security note

Live credentials — database password, JWT secret, Supabase service-role key, Groq key, Hugging Face tokens, Brevo key and a Gmail app password — were pasted into a development chat transcript during this work. They should be treated as disclosed and rotated. This is a process finding, not a code defect.

11. Testing and Continuous Integration

11.1 Test suites

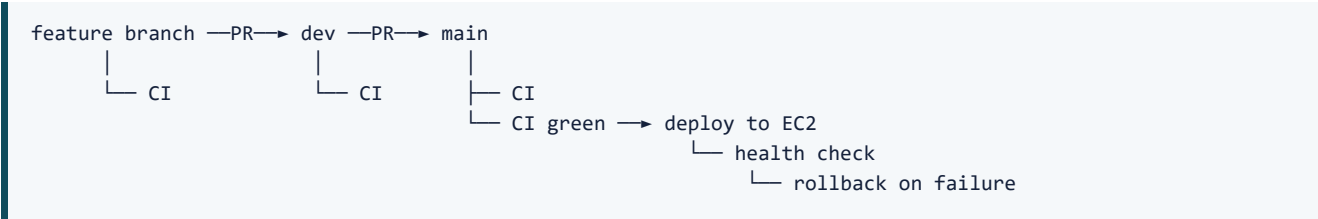
Suite	Tests	Requires database	Command
Backend unit	52	No	<code>python scripts/run_tests.py --unit</code>
Backend router (mocked)	9	No	included above
Backend integration	30	Yes	<code>python scripts/run_tests.py</code>
Frontend	62	No	<code>npm test</code>
Total	153		

Both runners print a per-group pass/fail table and exit non-zero on failure. The frontend runner also fails when zero tests execute, so a misconfigured reporter cannot report success while verifying nothing.

11.2 Verification of the tests themselves

Both suites were checked by mutation: the health-band defect was deliberately reintroduced and the suites were confirmed to fail — seven backend tests and three frontend tests, including the reachability guards written for that specific defect. A suite that stays green against known-broken code provides false assurance, so this check was treated as mandatory before reporting coverage.

11.3 Pipeline



Workflow	Jobs	Purpose
Backend CI	static-checks, unit-tests, integration-tests, ci-passed	Quality gate on every PR and push
Backend Deploy	deploy	Triggered by CI success on main only
Frontend CI	verify, audit	Typecheck, lint, tests, production build

Static checks compile every module, fail if `.env` or any private key is tracked, and fail on credential-shaped literals while ignoring documented placeholders. Deployment records the current commit, installs into the `virtualenv` `systemd` uses, byte-compiles before restarting, then polls health six times and rolls back to the recorded commit on failure.

12. Known Defects and Open Problems

This section records every defect known at the date of writing, including those not yet fixed. Each was verified against the running system; none is speculative.

12.1 Open — not yet fixed

HIGH Warehouse report is not warehouse-scoped

`build_warehouse_context(db, warehouse_id)` uses the identifier *only* to look up the warehouse name. Every data query inside it — assets, predictions, tickets, maintenance, costs — runs across the entire fleet.

```
total_assets = db.query(func.count(Asset.id)).scalar()    # no filter
... FROM pdm_batch_predictions WHERE status = 'ok'        # no filter
```

Impact: a report titled "LankaLogix - Colombo" (400 assets) describes all 850 across all three warehouses. Every figure in every warehouse report is fleet-wide.

Fix: add a warehouse filter to each query in that function. Not attempted because this area was designated out of scope for modification.

HIGH Asset and ticket summaries are not model-generated

Both Hugging Face Spaces return HTTP 404 — they no longer exist:

```
SharadaAbeywickrama/Asset_summery_generation    404
SharadaAbeywickrama/Ticket_summery_generation    404
```

Every call falls through to `_fallback_summary()`, a hand-written template. The template itself is honest — it is built from real asset fields — but the API reports it as model output regardless: `model_version` is hardcoded to "1.0" and the log records "✓ Summary generated" on both paths, so no consumer can tell which ran.

Aggravating factor: `HF_HUB_OFFLINE=1` is set in `main.py` and `.env`, which converts the 404 into a misleading "offline mode is enabled" error. The comment justifying that setting states only plain HTTP is used, but the summary services call `gradio_client`, which resolves through the Hub API.

Fix: restore or re-point the Spaces, and return a field indicating which path produced the summary.

HIGH Deployed instance drifts behind main

A running instance executes the nightly batch against the production database using whatever code it was last deployed with. This was observed directly: a batch run wrote health bands on the superseded capitalised scale and produced ten cost intervals excluding their own estimate, hours after both defects were fixed and merged.

Evidence was threefold — the distribution matched the old thresholds exactly (`Good = 8` is precisely the `≥ 70` count), `assets.health_band` was written correctly in the same run (so the writer had the earlier fix but not the later one), and the ten incoherent cost rows carried that run's timestamp.

Fix: the deploy workflow added in this review addresses the cause. It has not yet executed.

MEDIUM Cross-region database latency

Application in `ap-south-1`, database in `ap-southeast-2`. 154 ms per round-trip from EC2. Endpoints returning zero rows take 900–1,700 ms. Confirmed to be latency, not query volume: 2–4 SQL statements per request, connection pooling working correctly.

Fix: co-locate the two regions. Reducing round-trips in code has a much lower ceiling.

LOW Survival medians saturate the trained horizon

With event rates of 1.9%–17.1%, the Weibull fits are dominated by censored observations and median estimates cluster at the horizon limit for several components. Values at the ceiling are flagged `horizon_capped` and rendered as "180+" rather than as a precise count. This is a data property, not a defect, but it limits the practical value of the per-component day estimates.

12.2 Fixed during this review

Defect	Measured impact before fix
Four competing health-band definitions	602 of 850 assets banded differently on different screens; chatbot and dashboard disagreed
Warehouse report band scale uncalibrated	0 assets could ever be "healthy"; 621 of 850 reported critical; two permanently empty buckets
Decision-layer health tier used <code>≥ 80</code>	<code>healthy</code> tier unreachable — 0 of 850 assets; <code>conflict</code> inflated to 128, of which 120 were artifacts
All four <code>/predictions/*</code> endpoints returned 500 unconditionally	Response models declared fields the services never produce, so serialisation failed on every success
<code>healthPercent</code> populated from <code>criticality_score</code>	Mean error 29.4 points; 145 assets shown $\geq 70\%$ while actually poor or critical; one asset displayed 90.43% at a true health of 33.93
Cost interval excluded its own point estimate	Approximately half the fleet; 10 rows re-introduced later by the stale deployment
Cost fallback substituted a hand-written formula	Guesses stored beside real predictions with <code>status = "ok"</code> , indistinguishable afterwards
Empty request body produced 500	<code>validate_payload_fields</code> checked key presence, but Pydantic emits every optional key as <code>None</code>
PATCH semantics could not clear a field	Address and phone could be set but never cleared; API returned 200 regardless
Chatbot shipped debug text to users	Replies ended with "(Debug: Generic fallback hit)"
Cached warehouse data survived logout	Next user on a shared machine inherited the previous user's chatbot context
Conditional React hook in chart component	<code>useState</code> after an early return; React can throw "Rendered fewer hooks than expected". Component also redefined each render, discarding hover state
Unauthenticated model-inference endpoints	Any of 1,337 accounts could exhaust external quota
Notification preferences unscoped	Any user could create, read or delete another user's preferences

13. Technical Debt

13.1 Repository weight

The backend repository tracks 432 MB across 352 files. Roughly 260 MB is not required to run or build the system.

Category	Size	Detail
Superseded model artifacts	212 MB	32 files never loaded by runtime code. Two byte-identical copies of a 96.8 MB v5 regressor account for 194 MB alone.
Superseded dataset	48 MB	v10 CSV; the pipeline trains on v11
Runtime logs	190 KB	Four <code>uvicorn*.log</code> files committed to git
Dead application code	< 1 KB	<code>app/repositories/</code> queries <code>vehicles</code> and <code>vehicle_snapshots</code> , tables that do not exist in the current schema

Only four model artifacts are loaded at runtime: the survival bundle, the cost model, and the two v7 boosters. Thirteen further tracked files are 135-byte Git LFS pointers rather than real content, while `.gitattributes` contains no LFS filter rules — the tracking configuration was removed but the pointers remained. Nothing loads them, so there is no functional impact today.

13.2 Unused frontend modules

Eighteen components and service modules totalling approximately 104 KB are never imported. Four are superseded dashboard fragments left behind when that page was consolidated; one is a demonstration component.

Exclusion worth recording

`src/proxy.ts` appears unused to static analysis because nothing imports it. It is the Next.js 16 middleware convention file, loaded by the framework. Removing it would silently break request proxying. Any dead-code sweep on a Next.js project must exclude the framework's convention filenames.

13.3 Deprecated but retained

Twenty endpoints remain marked deprecated. The codebase convention is to deprecate rather than delete, preserving compatibility for any external consumer not visible in either repository.

13.4 Configuration debt

- `nixpacks.toml`, `railway.toml` and `render.yaml` configure three deployment platforms not in use
- Two near-identical 7.5 MB SQL seed dumps predate the fleet rebuild scripts
- 77 unused-variable lint warnings remain; eslint errors are at zero and the check is blocking

14. Recommendations

#	Action	Effort	Rationale
1	Deploy the merged main branch to EC2	Minutes	The instance is executing superseded code against production data nightly. Every other fix is provisional until this is done.
2	Rotate all disclosed credentials	Hours	Keys were exposed in a development transcript.
3	Scope <code>build_warehouse_context</code> by warehouse	Half a day	Every warehouse report currently describes the whole fleet under one site's name.
4	Restore the two summary Spaces, or label the fallback	Hours	The API asserts model provenance for template output.
5	Co-locate EC2 with Supabase in <code>ap-southeast-2</code>	1–2 days	Largest single performance gain available: 3–5× on request latency.
6	Remove the 260 MB of superseded artifacts	Hours	Clone and CI checkout time; history rewrite needed to reclaim fully.
7	Introduce a staging environment	2–3 days	<code>dev</code> is tested but never deployed, so <code>main</code> is the first contact with production infrastructure.
8	Extend test coverage to mutating endpoints	Days	Current tests probe GET comprehensively; create, update and delete paths are largely unverified.

Appendix A — Verification Method

Figures in this document were produced as follows.

Claim type	Method
Row counts, distributions	Direct SQL against the production database on 13 August 2026
Model metrics	Read from each model's own training log JSON, not recalled
Feature schema	Read from the deployed boosters' <code>feature_name()</code> metadata
Endpoint counts	Enumerated from the live FastAPI route table
Endpoint health	Every GET probed across three roles via TestClient
Dead code	Import-graph analysis, cross-checked by name search
Test counts	Executed, not counted from source
External dependencies	HTTP requests to each configured host

Analysis errors encountered and corrected

Several automated checks produced incorrect results before being corrected. They are recorded because they bear on how much confidence the figures deserve.

Error	Correction
Dead-endpoint scan matched only paths beginning with <code>/</code>	Missed four live <code>\${API_BASE_URL}/path</code> calls; pattern widened, count fell from 15 to 8
Auth scan did not recognise <code>require_super_admin</code>	<code>/batch-predictions/run</code> wrongly reported as ungated
Unbounded-query scan searched backwards only	Missed <code>.limit()</code> on a later line; 49 findings reduced to 31, most remaining benign
Frontend dead-code analysis flagged <code>src/proxy.ts</code>	Framework convention file; would have broken middleware
Credential scan matched a documented placeholder	<code>gsk_xxxxxxx</code> in help text; placeholder filter added and re-verified against a realistic key
Frontend test runner reported success on zero tests	Results file path failed on Windows; runner now treats zero tests as failure

Every quantitative claim in this document was measured. Where a figure could not be verified — for example the behaviour of the deploy workflow, which has not yet executed — that is stated explicitly rather than estimated.

Appendix B — Complete Technology Stack

B.1 Backend runtime and framework

Package	Version	Role in the system
fastapi	0.128.1	HTTP framework, dependency injection, OpenAPI generation
uvicorn	0.40.0	ASGI server, single worker under systemd
sqlalchemy	2.0.46	ORM and query construction
alembic	1.18.3	Schema migration tooling
psycopg / psycopg2-binary	3.3.2 / 2.9.11	PostgreSQL drivers
apscheduler	3.11.2	In-process 24-hour batch scheduler
pydantic-settings	2.12.0	Typed configuration from environment
python-jose	3.5.0	JWT encode and decode
passlib / bcrypt	1.7.4 / 4.0.1	Password hashing and verification
python-multipart	0.0.22	Multipart uploads for asset images
email-validator	2.3.0	Address validation at the schema boundary

B.2 Machine learning and numerical

Package	Version	Role
lightgbm	4.6.0	v7 classifier and regressor; cost-model quantile bounds
catboost	1.2.10	Cost-model point estimate
lifelines	0.30.3	Weibull AFT survival models
xgboost	3.2.0	Legacy v6 artifacts; ticket models on the HF Space
shap	0.51.0	Explainability (LightGBM native <code>pred_contrib</code> used in the hot path)
scikit-learn	1.6.1	TF-IDF vectoriser for the knowledge base
pandas / numpy	2.3.3 / 2.4.3	Feature frame construction
joblib	1.5.3	Model artifact serialisation
nltk	3.9.4	Text preprocessing

B.3 External services

Package	Version	Service
groq / langchain-groq	0.37.1 / 1.1.2	Llama 3.3 70B for reports and chatbot
langchain-core	1.4.0	Prompt and message abstractions
huggingface_hub	1.8.0	Space resolution for ticket models
supabase	2.28.3	Storage and Admin API client
openai	1.59.3	Client library retained for compatibility
requests	2.33.0	Direct HTTP to Gradio Spaces and Brevo
reportlab	—	Server-side PDF rendering

B.4 Frontend

Package	Version	Role
next	16.2.9	App Router, server components, build pipeline
react / react-dom	19.2.3	UI runtime
typescript	5.x	Static typing, strict mode
tailwindcss	4.1.18	Utility-first styling
recharts	3.6.0	All dashboard and analytics charts
lucide-react	—	Icon set
shadcn/ui + Radix	—	Accessible dialog, select, tabs, badge primitives
vitest + Testing Library	—	Unit and component tests
eslint / eslint-config-next	9.x / 16.1.1	Linting, React hook rules

B.5 Infrastructure

Component	Provider	Detail
Application host	AWS EC2	ap-south-1 , Ubuntu, systemd unit <code>predictix</code>
Reverse proxy	nginx	Fronts uvicorn on port 8000
Database	Supabase PostgreSQL	ap-southeast-2 , session pooler on 5432
Object storage	Supabase Storage	Asset images, generated reports
Frontend hosting	Vercel	Automatic deployment from the repository
Email	Brevo	Transactional notifications
Model hosting	Hugging Face Spaces	Ticket categorisation and priority
CI/CD	GitHub Actions	Three workflows across two repositories

Appendix C — Complete Endpoint Reference

All 140 HTTP endpoints as registered by the running application, grouped by router. Deprecated routes are marked; they remain mounted for compatibility with any consumer outside these two repositories.

/admin-dashboard (1 endpoints)

Method	Path	Purpose
GET	/admin-dashboard/summary	

/asset-assignments (3 endpoints)

Method	Path	Purpose
GET	/asset-assignments/	
POST	/asset-assignments/	
DELETE	/asset-assignments/{assignment_id}	

/asset-reports (2 endpoints)

Method	Path	Purpose
GET	/asset-reports/dummy/pdf	[Dev/QA only] Render a PDF with hardcoded dummy data
POST	/asset-reports/{asset_id} DEPRECATED	[Deprecated] Server-rendered asset report

/asset-summaries (3 endpoints)

Method	Path	Purpose
GET	/asset-summaries/by-asset/{asset_id}	
POST	/asset-summaries/generate	
GET	/asset-summaries/health	

/assets (13 endpoints)

Method	Path	Purpose
GET	/assets/	
POST	/assets/	
GET	/assets/analytics	
GET	/assets/count	
GET	/assets/dropdown	Lightweight asset list for dropdowns
GET	/assets/stats	
DELETE	/assets/{asset_id}	
GET	/assets/{asset_id}	
PUT	/assets/{asset_id}	
PATCH	/assets/{asset_id}/assign	
GET	/assets/{asset_id}/component-rul	DEPRECATED [Deprecated] Use GET /survival/{asset_id} instead
POST	/assets/{asset_id}/send-service-reminder	Send a service reminder email to the assigned user (admin
PATCH	/assets/{asset_id}/status	

/auth (4 endpoints)

Method	Path	Purpose
POST	/auth/google-login	
POST	/auth/login	
POST	/auth/login/select-warehouse	
GET	/auth/warehouses	

/batch-predictions (4 endpoints)

Method	Path	Purpose
GET	/batch-predictions/	Latest cached PDM predictions for all assets
POST	/batch-predictions/run	Trigger a full batch prediction run for all assets
POST	/batch-predictions/run/{asset_id}	Trigger a fresh prediction for one asset
GET	/batch-predictions/{asset_id}	Latest cached PDM prediction for one asset

/chatbot (2 endpoints)

Method	Path	Purpose
POST	/chatbot/agent	
POST	/chatbot/ask	DEPRECATED

/departments (5 endpoints)

Method	Path	Purpose
GET	/departments/	
POST	/departments/	
DELETE	/departments/{department_id}	
GET	/departments/{department_id}	
PUT	/departments/{department_id}	

/faqs (4 endpoints)

Method	Path	Purpose
GET	/faqs/	
POST	/faqs/	
DELETE	/faqs/{faq_id}	
PUT	/faqs/{faq_id}	

/maintenance (6 endpoints)

Method	Path	Purpose
GET	/maintenance/	
POST	/maintenance/	
POST	/maintenance/log-maintenance/{asset_id}	Record a completed service against an asset
DELETE	/maintenance/{event_id}	
GET	/maintenance/{event_id}	
PUT	/maintenance/{event_id}	

/notification-preferences (3 endpoints)

Method	Path	Purpose
GET	/notification-preferences/	
POST	/notification-preferences/	
DELETE	/notification-preferences/{preference_id}	

/notifications (7 endpoints)

Method	Path	Purpose
GET	/notifications	
POST	/notifications	
PUT	/notifications/mark-all-read	
GET	/notifications/unread	
DELETE	/notifications/{notification_id}	
GET	/notifications/{notification_id}	
PUT	/notifications/{notification_id}/mark-read	

/predictions (7 endpoints)

Method	Path	Purpose
POST	/predictions/classification	
GET	/predictions/cost/{asset_id}	
POST	/predictions/full	
GET	/predictions/health	
POST	/predictions/health-score	
POST	/predictions/regression	
GET	/predictions/ticket/{ticket_id}	

/profiles (9 endpoints)

Method	Path	Purpose
GET	/profiles/	
GET	/profiles/me	
PUT	/profiles/me	
GET	/profiles/me/assets	
POST	/profiles/me/avatar	
GET	/profiles/me/colleagues	
GET	/profiles/me/stats	
GET	/profiles/{profile_id}	
PUT	/profiles/{profile_id}	

/reports (6 endpoints)

Method	Path	Purpose
GET	/reports/	
POST	/reports/	
POST	/reports/render-pdf	
DELETE	/reports/{report_id}	
GET	/reports/{report_id}	
PUT	/reports/{report_id}	

/root (1 endpoints)

Method	Path	Purpose
GET	/	

/sensor-readings (2 endpoints)

Method	Path	Purpose
POST	/sensor-readings/	
GET	/sensor-readings/asset/{asset_id}	

/survival (2 endpoints)

Method	Path	Purpose
GET	/survival/{asset_id}	
GET	/survival/{asset_id}/{component}	

/ticket-attachments (3 endpoints)

Method	Path	Purpose
GET	/ticket-attachments/	
POST	/ticket-attachments/	
DELETE	/ticket-attachments/{attachment_id}	

/ticket-comments (3 endpoints)

Method	Path	Purpose
GET	/ticket-comments/	
POST	/ticket-comments/	
DELETE	/ticket-comments/{comment_id}	

/ticket-status-history (2 endpoints)

Method	Path	Purpose
GET	/ticket-status-history/	
POST	/ticket-status-history/	

/ticket-summaries (3 endpoints)

Method	Path	Purpose
GET	/ticket-summaries/by-ticket/{ticket_id}	
POST	/ticket-summaries/generate	
GET	/ticket-summaries/health	

/tickets (14 endpoints)

Method	Path	Purpose
GET	/tickets/	
POST	/tickets/	
POST	/tickets/categorize	
GET	/tickets/mine	
POST	/tickets/mine	
DELETE	/tickets/mine/{ticket_id}	
PUT	/tickets/mine/{ticket_id}	
GET	/tickets/paginated	
POST	/tickets/preview	
POST	/tickets/prioritize	Classify ticket priority
GET	/tickets/status-counts	
DELETE	/tickets/{ticket_id}	
GET	/tickets/{ticket_id}	
PUT	/tickets/{ticket_id}	

/user (9 endpoints)

Method	Path	Purpose
GET	/user/tickets	
POST	/user/tickets	
POST	/user/tickets/preview	
GET	/user/tickets/stats	
GET	/user/tickets/{ticket_id}	
PUT	/user/tickets/{ticket_id}	
POST	/user/tickets/{ticket_id}/attachments	
GET	/user/tickets/{ticket_id}/comments	
POST	/user/tickets/{ticket_id}/comments	

/users (5 endpoints)

Method	Path	Purpose
GET	/users/	
POST	/users/	
DELETE	/users/{user_id}	
PUT	/users/{user_id}	
GET	/users/{user_id}/assets	

/vehicle-predictions (1 endpoints)

Method	Path	Purpose
POST	/vehicle-predictions/{asset_id} DEPRECATED	[Deprecated] Use POST /batch-predictions/run/{asset_id} in

/warehouse-dashboard (7 endpoints)

Method	Path	Purpose
POST	/warehouse-dashboard/chat-route	
GET	/warehouse-dashboard/departments-overview	
GET	/warehouse-dashboard/generate-report	
GET	/warehouse-dashboard/maintenance-schedule	
POST	/warehouse-dashboard/notify-print	
GET	/warehouse-dashboard/summary	
GET	/warehouse-dashboard/survival	

/warehouses (3 endpoints)

Method	Path	Purpose
GET	/warehouses/	
POST	/warehouses/	
GET	/warehouses/{warehouse_id}	

/warmup (1 endpoints)

Method	Path	Purpose
POST	/warmup/inference-space	

Appendix D — Data Flows

D.1 Nightly prediction batch

The dominant write path. Runs on a 24-hour interval inside the application process, calling the batch service directly as Python rather than over HTTP.

```

APScheduler (interval 24h)
|
▼
run_batch_for_all_assets(db, clf, reg, cost_bundle)
|
├─ SELECT active assets                — 1 round-trip
├─ SELECT latest sensor reading per asset — 1 round-trip (LATERAL)
|
├─ build 850 feature dicts              in memory
├─ clf.predict_proba_positive(frame)    1 vectorised call
├─ reg.predict(frame) + shap_top_factors(frame) 1 vectorised call
├─ _compute_health_score per asset
├─ _estimate_cost per asset → NULL when the bundle is absent
├─ build_decision per asset → tier, display mode, copy
|
├─ UPSERT pdm_batch_predictions (one row per asset)
└─ UPDATE assets.health_band (from the same score)

```

Two model calls serve the whole fleet. The final step is what keeps the assets list and the prediction row in agreement by construction.

D.2 Asset detail page load

```

Browser: /admin/assets → selects an asset
|
▼
getAssetDetail(assetId)          six requests issued in parallel
├─ GET /assets/{id}              asset record          (rejects on failure)
├─ GET /batch-predictions/{id}   prediction            (null on failure)
├─ GET /survival/{id}            component RUL        (null on failure)
├─ GET /maintenance/?asset_id=   service history      ([] on failure)
├─ GET /tickets/?asset_id=       related tickets       ([] on failure)
├─ GET /asset-assignments/?asset_id= assignment history    ([] on failure)
|
▼
AssetDetailsPanel renders. A missing prediction shows "Not scored",
never a substituted number.

```

Only the asset request is allowed to fail the whole load; the history calls degrade to empty so a missing ticket list cannot blank the page.

D.3 Authentication

```

POST /auth/login {email, password}
├─ profile lookup by email
├─ bcrypt verify against profiles.meta.password_hash
├─ regular user → JWT issued immediately
├─ super admin → requires_warehouse_selection + selection_token
│   └─
│       └─ POST /auth/login/select-warehouse
│           └─ JWT issued carrying the chosen warehouse_id

```

Every subsequent request:

```

Authorization: Bearer
├─ decode and validate signature
├─ re-read the profile row — 1 round-trip, every request
├─ reject if status is not active (immediate deactivation)
├─ override warehouse_id from the token for super admins

```

D.4 Ticket creation with model assistance

```

User submits a ticket
├─ categorize_ticket_text(title, description)
│   └─ HF Space (Gradio) → predicted_category + confidence
├─ predict_ticket_priority(text)
│   └─ HF Space → predicted_priority
├─ INSERT ticket with predicted_* and final_* fields
├─ NotificationService → in-app rows + Brevo email
├─ WebSocket push to /ws/notifications/{user_id}

```

Predicted and final values are stored in separate columns, so a human override never destroys what the model said.

D.5 Warehouse report generation (RAG)

```

POST report request
├─ build_warehouse_context(db, warehouse_id)
│   └─ live SQL: fleet composition, health bands, maintenance,
│       tickets, costs, survival summary
├─ KB vector store (TF-IDF over 15 documents)
│   └─ retrieve passages: ISO 55000, SMRP benchmarks, OEM intervals
├─ KB annotator (deterministic, in code)
│   └─ SHAP thresholds, band interpretations, benchmark alerts
├─ prompt assembly: SQL figures + KB context + "reuse these numbers verbatim"
├─ Groq Llama 3.3 70B → narrative sections
├─ client-side PDF assembly, or POST /reports/render-pdf server-side

```

D.6 Chatbot query

```
POST /chatbot/agent {question, frontend_context}
|
| table selector      → which allow-listed tables are relevant
| SQL generator      → parameterised query over those tables only
| execute            → rows
| summariser         → natural-language answer
| fallback           → contact-admin reply, logged, no debug text
```

`frontend_context` carries cached dashboard and asset snapshots from local storage, which are cleared on both logout and session establishment so one user's figures cannot leak into another's session.

Appendix E — Applied Optimisations

Each entry below was measured before and after, not assumed.

E.1 Query and payload

Optimisation	Effect
Vectorised model inference	Two booster calls serve 850 assets instead of 1,700 individual calls
LATERAL join for latest sensor reading	One query replaces one-per-asset lookups
Merged duplicate <code>COUNT in /profiles/me/stats</code>	3 → 2 SQL statements; one round-trip saved per call
Batched warehouse-name resolution	Removed an N+1 in the user-assets path
Batched asset-name resolution in ticket lists	One <code>IN</code> query per page instead of per row
Bounded <code>/profiles/me/colleagues</code>	140 KB → 2.3 KB for a preview request
Trimmed list projection (<code>AssetListOut</code>)	List view transfers the columns it renders, not all 34
Server-side pagination, 50 rows per page	Replaced whole-fleet transfers
Removed a redundant profile re-fetch	One round-trip saved on <code>/me/colleagues</code>

E.2 Caching

Cache	Scope and lifetime
Dashboard payload cache	Server-side, 1,800 s TTL
Asset list and count cache	Browser, 15 s TTL, keyed by query string
Reference data cache	Departments and warehouses, process-lifetime
HF Space warmup cooldown	60 s process-wide; several tabs produce one ping

E.3 Database

107 indexes exist in the public schema, 36 of them added by the migration series in [docs/migrations/](#). Largest relations by total size:

Table	Size
sensor_readings	30 MB
maintenance_events	18 MB
pdm_batch_predictions	5.2 MB
profiles	3.8 MB
assets	2.0 MB

Connection pooling is configured with `pool_size=5`, `max_overflow=3` and `pool_recycle=1800`, deliberately below the pooler's connection ceiling. Pooling was verified working: repeated sessions reuse the same PostgreSQL backend process.

An optimisation deliberately not applied

`pool_pre_ping=True` costs approximately one extra round-trip per request and was measured at roughly 190 ms on a warm pool. It was retained rather than removed: it is the guard against stale connections through the Supabase pooler on a cross-region link, and trading a correctness guarantee for latency is the wrong exchange while the real cause is geography. Co-location addresses the cost properly.

E.4 Frontend

Optimisation	Effect
Parallel detail fetch	Six requests concurrently rather than sequentially
Debounced search (400 ms, 2-character minimum)	Prevents a fleet-wide query per keystroke
Cancellation guards in effects	A slow earlier response cannot overwrite a newer one
Component hoisted out of render	Chart node is one stable type; hover state no longer discarded each render
Bounded notification bell	30 items, with full history behind a separate view
Team preview capped	8 members loaded instead of a full department

Appendix F — Frontend Structure in Detail

F.1 Route inventory

Next.js App Router with three route groups. Groups are organisational and do not appear in the URL.

Route	Group	Audience	Purpose
/	—	Public	Landing page
/login	(auth)	Public	Credential and Google sign-in; fires Space warmup
/login/callback	(auth)	Public	Google OAuth return, exchanges for a JWT
/admin/dashboard	(admin)	Admin	Fleet KPIs, health and cost trends, alerts, AI summary
/admin/assets	(admin)	Admin	Paginated fleet list with detail panel and analytics
/admin/tickets	(admin)	Admin	Ticket queue, charts, assignment
/admin/users	(admin)	Admin	Account management and asset assignment
/admin/warehouse	(admin)	Admin	Warehouse dashboard and survival analysis
/admin/warehouse/report	(admin)	Admin	Generated warehouse report view
/admin/profile , /admin/settings	(admin)	Admin	Self-service profile and preferences
/user/dashboard	(user)	User	Assigned assets, own tickets, team preview
/user/assets	(user)	User	Assigned assets with read-only detail; fleet search
/user/tickets	(user)	User	Own tickets, creation, comments
/user/users	(user)	User	Team directory
/user/profile , /user/settings	(user)	User	Self-service profile and preferences
/help-desk	—	All	FAQ browser with admin editing
/privacy-policy , /terms-of-service	—	Public	Legal pages
/test-loader , /theme-preview	—	—	Development pages, present in the production build

F.2 Component inventory

Directory	Components	Contents
<code>components/admin/</code>	38	assets, tickets, users, warehouse, dashboard, dialogs
<code>components/ui/</code>	18	shadcn/Radix primitives: dialog, select, tabs, badge, table
<code>components/navigation/</code>	10	Navbars, notification bell and dialogs, route loader
<code>components/user/</code>	4	User-role asset and ticket views
<code>components/background/</code>	3	Decorative backgrounds
<code>components/auth/</code> , <code>common/</code> , <code>chat/</code> , <code>loading/</code>	6	Login cards, section wrappers, floating chatbot, loader

F.3 Library modules

Module	Responsibility
<code>apiClient.ts</code>	Single fetch wrapper; attaches JWT, raises typed <code>ApiError</code>
<code>authService.ts</code>	Login, warehouse selection, token and user storage, logout and cache clearing
<code>healthBands.ts</code>	The one frontend definition of band thresholds and colours
<code>dashboardService.ts</code>	Admin dashboard payload types and fetch
<code>warehouseService.ts</code>	Warehouse dashboard and survival data
<code>ticketService.ts</code> , <code>userTicketService.ts</code>	Admin and user ticket operations
<code>userService.ts</code>	Account management, assignment and unassignment
<code>api/notificationsApi.ts</code>	Notification list, read state, deletion
<code>api/userProfileApi.ts</code>	Self-service profile, assets, stats, colleagues
<code>assetPdfExport.ts</code>	Client-side asset report, server render fallback
<code>professionalPdfExport.ts</code>	Warehouse report PDF composition
<code>supabaseBrowserClient.ts</code>	Browser Supabase client for OAuth
<code>customToast.ts</code> , <code>utils.ts</code>	Notifications and class merging

F.4 State and rendering conventions

- Client components (`"use client"`) wherever interaction or browser storage is required; the rest render on the server
- Local `useState` and `useEffect` only — no global store
- Async effects carry a `cancelled` flag so a superseded response cannot overwrite newer state
- Loading handled with skeletons rather than spinners where layout is known
- Errors surfaced through toasts carrying the server's `detail` message
- Nullable numerics render an em-dash placeholder, never a substituted zero

F.5 Authentication in the browser

Storage key	Contents
<code>predictix.access_token</code>	JWT (legacy <code>token</code> key also read)
<code>predictix.user</code>	Serialised profile; corrupt JSON yields null rather than throwing
<code>predictix.user.*</code>	Role, email, id, name, warehouse id and name
<code>predictix.cached_dashboard_data</code>	Dashboard snapshot, also read by the chatbot
<code>predictix.cached_asset_stats</code>	Asset statistics snapshot
<code>predictix.cached_asset_analytics</code>	Asset analytics snapshot

The three cached snapshots are cleared on logout *and* on session establishment, because they feed the chatbot's context.

Appendix G — Backend Structure in Detail

G.1 Package layout

app/	
main.py	application factory, model loading, scheduler, router mounting
deps.py	get_db, get_current_user, require_user/admin/super_admin
models.py	SQLAlchemy ORM definitions
core/config.py	typed settings from the environment
db/session.py	engine and session factory, pool configuration
routers/	29 routers, thin: validation and scoping only
schemas/	Pydantic request and response models
services/	business logic: notifications, health bands, reminders, maintenance classification, PDF rendering, caches
ai/	
models/	trained artifacts and training notebooks
services/	batch pipeline, prediction, survival, cost, summaries
agent/	chatbot tools, actions, memory
agents/	warehouse report generation
kb/	knowledge documents, TF-IDF store, annotator
tests/	unit, qa_automated, integration
docs/migrations/	ordered SQL migrations
scripts/	operational entry points

G.2 Layering rule

Routers validate input, resolve the caller and enforce scoping, then delegate. Business logic lives in `services/` and `ai/services/`. This is why the scheduler can invoke the batch pipeline directly as Python without an HTTP round-trip, and why the same logic serves both the scheduled run and the manual refresh.

G.3 Shared definition modules

Module	Single definition it owns
<code>services/health_bands.py</code>	Band thresholds, <code>band_for</code> , and SQL <code>CASE</code> generation so Python and set-based paths cannot diverge
<code>services/maintenance_classification.py</code>	Planned versus unplanned event types, following EN 13306 and SMRP
<code>ai/services/pdm_decision_service.py</code>	Tier reconciliation and display mode
<code>ai/services/lgb_model_adapter.py</code>	Booster loading, categorical vocabulary, frame construction, SHAP

Each exists because the same concept had previously been re-implemented in several places with different constants, which is how the same asset came to be described differently on different screens.

G.4 Operational scripts

Script	Purpose
<code>scripts/run_tests.py</code>	Test runner with a per-group pass/fail table
<code>scripts/apply_migration.py</code>	Applies a numbered migration; dry-run by default
<code>scripts/recompute_decisions.py</code>	Recomputes decision fields after a logic change; dry-run by default
<code>scripts/rebuild_fleet.py</code>	Regenerates the fleet from v11 trajectories
<code>scripts/seed_faqs.py</code>	Loads the help-desk knowledge base

G.5 Migration series

Migration	Change
002	PdM decision-layer columns on <code>pdm_batch_predictions</code>
003	Prediction history table
004, 006, 008	Index additions, including asset sort columns
005	Table privileges for report tables
007	Survival model type enum
009	Parking slot column and health-percentage check constraints
010	Aligns <code>health_status</code> with the canonical bands
011	Clamps incoherent cost intervals to contain their estimate

Migrations 010 and 011 are idempotent: their `WHERE` clauses make a second run a no-op, which matters because both had to be re-applied after the stale deployment overwrote their effect.